# Celebal Excellence Internship

[Week – 2 Task – E commerce sales database]

Title: CEI – week2 task submission

Name: Anushka Manoj Patil

College name: Dr. D. Y. Patil Pimpri, Pune

## 1. Table creation:

- Creating first table CUSTOMERS

  Query:

```
CREATE TABLE CUSTOMERS(
CUSTOMER_ID INT PRIMARY KEY,
FIRST_NAME VARCHAR(50) NOT NULL,
LAST_NAME VARCHAR(50) NOT NULL,
EMAIL VARCHAR(100) UNIQUE NOT NULL,
CITY VARCHAR(50) NOT NULL,
STATE VARCHAR(50) NOT NULL,
JOIN_DATE DATE NOT NULL,
IS_PREMIUM BIT DEFAULT 0
);
```

  For index:

```
CREATE INDEX IDX_CUSTOMERS_CITY ON CUSTOMERS(CITY);
CREATE INDEX IDX_CUSTOMERS_STATE ON CUSTOMERS(STATE);
```

- Creating second table PRODUCTS

  Query:

```
CREATE TABLE PRODUCTS(
PRODUCT_ID INT PRIMARY KEY,
PRODUCT_NAME VARCHAR(50) NOT NULL,
CATEGORY VARCHAR(50) NOT NULL,
BRAND VARCHAR(50),
UNIT_PRICE DECIMAL(10,2) NOT NULL
CHECK (UNIT_PRICE > 0),
STOCK_QTY INT NOT NULL DEFAULT 0
CHECK (STOCK_QTY >= 0));
```

For index:

```
CREATE INDEX IDX_PRODUCTS_CATEGORY ON
PRODUCTS(CATEGORY);
```

- Creating third table : ORDERS

  Query:

```
CREATE TABLE ORDERS(
ORDER_ID INT PRIMARY KEY,
CUSTOMER_ID INT NOT NULL,
ORDER_DATE DATE NOT NULL,
ORDER_STATUS VARCHAR(20) NOT NULL DEFAULT 'Pending'
CHECK (ORDER_STATUS IN('Pending', 'Shipped', 'Delivered',
'Cancelled')),
TOTAL_AMOUNT DECIMAL(12,2) NOT NULL
CHECK (TOTAL_AMOUNT >= 0),
FOREIGN KEY (CUSTOMER_ID) REFERENCES
CUSTOMERS(CUSTOMER_ID));
```

- Creating fourth table : ORDERS_ITEMS

  Query:

```
CREATE TABLE ORDER_ITEMS(
ITEM_ID INT PRIMARY KEY,
ORDER_ID INT NOT NULL,
PRODUCT_ID INT NOT NULL,
QUANTITY INT NOT NULL
CHECK (QUANTITY > 0),
UNIT_PRICE DECIMAL(10,2) NOT NULL
CHECK (UNIT_PRICE > 0),
```

```
DISCOUNT_PCT DECIMAL(5,2) DEFAULT 0
CHECK (DISCOUNT_PCT BETWEEN 0 AND 100),
FOREIGN KEY(ORDER_ID) REFERENCES ORDERS (ORDER_ID),
FOREIGN KEY (PRODUCT_ID) REFERENCES
PRODUCTS(PRODUCT_ID));
FOREIGN KEY (CUSTOMER_ID) REFERENCES
CUSTOMERS(CUSTOMER_ID));
```

## 2. Inserting values in table

- Inserting values in CUSTOMERS:

Query:

```
INSERT INTO customers VALUES
(101,'Aarav','Sharma','aarav.s@email.com','Mumbai','Maharashtra
','2024-01-15',1),
(102,'Priya','Patel','priya.p@email.com','Ahmedabad','Gujarat','20
24-02-20',0),
(103,'Rohan','Gupta','rohan.g@email.com','Delhi','Delhi','2024-03-
10',1),
(104,'Sneha','Reddy','sneha.r@email.com','Hyderabad','Telangana'
,'2024-04-05',0),
(105,'Vikram','Singh','vikram.s@email.com','Jaipur','Rajasthan','20
24-05-12',1),
(106,'Ananya','Iyer','ananya.i@email.com','Chennai','Tamil
Nadu','2024-06-18',0),
(107,'Karan','Mehta','karan.m@email.com','Pune','Maharashtra','2
024-07-22',1),
(108,'Divya','Nair','divya.n@email.com','Kochi','Kerala','2024-08-
30',0);
```

- Inserting values in PRODUCTS

Query:

```
INSERT INTO products VALUES
(201,'Wireless Earbuds','Electronics','BoAt',1499.00,250),
```

(202,'Cotton T-Shirt','Clothing','Levis',799.00,500),
(203,'Smart Watch','Electronics','Noise',2999.00,150),
(204,'Running Shoes','Clothing','Nike',4599.00,120),
(205,'Bluetooth Speaker','Electronics','JBL',3499.00,200),
(206,'Bedsheet Set','Home','Spaces',1299.00,300),
(207,'Laptop Stand','Electronics','AmazonBasics',899.00,180),
(208,'Cushion Covers (Set)','Home','HomeCenter',599.00,400);

- Inserting values into ORDERS

  Query:

(1001,101,'2024-08-01','Delivered',4498.00),
(1002,102,'2024-08-03','Delivered',799.00),
(1003,103,'2024-08-05','Shipped',7498.00),
(1004,101,'2024-08-10','Delivered',3499.00),
(1005,104,'2024-08-12','Cancelled',2999.00),
(1006,105,'2024-08-15','Delivered',5898.00),
(1007,106,'2024-08-18','Pending',1299.00),
(1008,103,'2024-08-20','Delivered',899.00),
(1009,107,'2024-08-25','Shipped',6098.00),
(1010,108,'2024-08-28','Delivered',1598.00);

- Inserting values into ORDER_ITEMS

  Query:

(5001,1001,201,2,1499.00,0),
(5002,1001,207,1,899.00,10),
(5003,1002,202,1,799.00,0),
(5004,1003,203,1,2999.00,0),
(5005,1003,204,1,4599.00,5),
(5006,1004,205,1,3499.00,0),
(5007,1005,203,1,2999.00,0),
(5008,1006,201,1,1499.00,10),
(5009,1006,204,1,4599.00,5),
(5010,1007,206,1,1299.00,0),
(5011,1008,207,1,899.00,0),
(5012,1009,205,1,3499.00,0),
(5013,1009,208,2,599.00,15),
(5014,1010,206,1,1299.00,0),

(5015,1010,208,1,599.00,0);

# Section – A

**Q1.** Write a query to display all columns and rows from the customer's table.

Query:

```
SELECT * FROM CUSTOMERS;
```

Result:

| | CUSTOMER_ID | FIRST_NAME | LAST_NAME | EMAIL | CITY | STATE | JOIN_DATE | IS_PREMIUM |
|---|---|---|---|---|---|---|---|---|
| 1 | 101 | Aarav | Sharma | aarav.s@email.com | Mumbai | Maharashtra | 2024-01-15 | 1 |
| 2 | 102 | Priya | Patel | priya.p@email.com | Ahmedabad | Gujarat | 2024-02-20 | 0 |
| 3 | 103 | Rohan | Gupta | rohan.g@email.com | Delhi | Delhi | 2024-03-10 | 1 |
| 4 | 104 | Sneha | Reddy | sneha.r@email.com | Hyderabad | Telangana | 2024-04-05 | 0 |
| 5 | 105 | Vikram | Singh | vikram.s@email.com | Jaipur | Rajasthan | 2024-05-12 | 1 |
| 6 | 106 | Ananya | Iyer | ananya.i@email.com | Chennai | Tamil Nadu | 2024-06-18 | 0 |
| 7 | 107 | Karan | Mehta | karan.m@email.com | Pune | Maharashtra | 2024-07-22 | 1 |
| 8 | 108 | Divya | Nair | divya.n@email.com | Kochi | Kerala | 2024-08-30 | 0 |

Observation:

This query returns all the records from the customer table.

**Q2.** Retrieve only the first_name, last_name, and city of all customers.

Query:

```
SELECT FIRST_NAME, LAST_NAME,CITY FROM CUSTOMERS;
```

Result:

| | FIRST_NAME | LAST_NAME | CITY |
|---|---|---|---|
| 1 | Aarav | Sharma | Mumbai |
| 2 | Priya | Patel | Ahmedabad |
| 3 | Rohan | Gupta | Delhi |
| 4 | Sneha | Reddy | Hyderabad |
| 5 | Vikram | Singh | Jaipur |
| 6 | Ananya | Iyer | Chennai |
| 7 | Karan | Mehta | Pune |
| 8 | Divya | Nair | Kochi |

Observation:

This query returns first name, last name and city from table CUSTOMER.

**Q3.** List all unique categories available in the products table.

Query:

```
SELECT DISTINCT CATEGORY FROM PRODUCTS;
```

Result:

| | CATEGORY |
|---|---|
| 1 | Clothing |
| 2 | Electronics |
| 3 | Home |

**Q4.** Identify the Primary Key of each table in the schema. Explain why a Primary Key must be unique and NOT NULL.

Answer:

Primary key of each table:

CUSTOMERS : CUSTOMER_ID

PRODUCTS : PRODUCT_ID

ORDERS : ORDER_ID

ORDER_ITEMS : ITEM_ID

The primary key is used to uniquely identify each record.

Hence every record is need to be UNIQUE and NOT NULL.

So no record can have the same value or null value as Primary key.

Conclusion:

Primary key is combination of UNIQUE+NOT NULL values.

**Q5.** What constraints are applied to the email column in the customers table? What would happen if you tried to insert a duplicate email?

Answer:

- ✓ Constraints applied to email column in the customers table are 'UNIQUE' and 'NOT NULL'.
- ✓ If we try to insert the duplicate email then it well throw the error 'violation of UNIQUE KEY constraint.' and that new email cannot be inserted into the table.

**Q6.** Try inserting a product with unit_price = -50. What happens and which constraint prevents it? Write both the INSERT statement and explain the error.

Answer:

- ✓ INSERT INTO PRODUCTS (PRODUCT_ID,PRODUCT_NAME,CATEGORY,BRAND,UNIT_PRICE,STOCK_QTY)

  VALUES (209,'USB cable','electronics','samsung',-50,100);

Result:

- ✓ This query will throw the error as 'The INSERT statement conflicted with'

  the CHECK constraint'
- ✓ The SQL server will not allow to insert the negative record in the PRODUCTS table

Reason:

- ✓ The constraint which prevents it is: CHECK constraint
- ✓ At the time of table creation we have used the CHECK constraint for unit_price record as 'CHECK(UNIT_PRICE > 0)'

Hence it will not allow the insertion of values lesser than 0

Conclusion:

By using the CHECK constraint we can restrict the values by specifying the boundries for it.

## Section – B

### Q7. Retrieve all orders with status = 'Delivered'.

Query:

```
SELECT * FROM ORDERS
WHERE ORDER_STATUS='Delivered';
```

Result:

| | ORDER_ID | CUSTOMER_ID | ORDER_DATE | ORDER_STATUS | TOTAL_AMOUNT |
|---|---|---|---|---|---|
| 1 | 1001 | 101 | 2024-08-01 | Delivered | 4498.00 |
| 2 | 1002 | 102 | 2024-08-03 | Delivered | 799.00 |
| 3 | 1004 | 101 | 2024-08-10 | Delivered | 3499.00 |
| 4 | 1006 | 105 | 2024-08-15 | Delivered | 5898.00 |
| 5 | 1008 | 103 | 2024-08-20 | Delivered | 899.00 |
| 6 | 1010 | 108 | 2024-08-28 | Delivered | 1598.00 |

Observation:

The query returns the record with Delivered status.

### Q8. Find all products in the 'Electronics' category with a unit_price greater than ₹2000.

Query:

```
SELECT * FROM PRODUCTS
WHERE CATEGORY = 'Electronics'
```

AND

UNIT_PRICE > 2000;

Result:

| | PRODUCT_ID | PRODUCT_NAME | CATEGORY | BRAND | UNIT_PRICE | STOCK_QTY |
|---|---|---|---|---|---|---|
| 1 | 203 | Smart Watch | Electronics | Noise | 2999.00 | 150 |
| 2 | 205 | Bluetooth Speaker | Electronics | JBL | 3499.00 | 200 |

Observation:

This query returns the records from “Electronics” category and having UNIT_PRICE greater than 2000

## **Q9.** List all customers who joined in the year 2024 and belong to the state 'Maharashtra'.

Query:

SELECT * FROM CUSTOMERS

WHERE JOIN_DATE BETWEEN '2024-01-01' AND '2024-12-31'

AND STATE = 'Maharashtra';

Result:

| | CUSTOMER_ID | FIRST_NAME | LAST_NAME | EMAIL | CITY | STATE | JOIN_DATE | IS_PREMIUM |
|---|---|---|---|---|---|---|---|---|
| 1 | 101 | Aarav | Sharma | aarav.s@email.com | Mumbai | Maharashtra | 2024-01-15 | 1 |
| 2 | 107 | Karan | Mehta | karan.m@email.com | Pune | Maharashtra | 2024-07-22 | 1 |

Observation:

This query returns the records from “Maharashtra” state and joined in year 2024

## **Q10.** Find all orders placed between '2024-08-10' and '2024-08-25' (inclusive) that are NOT cancelled.

Query:

```
SELECT * FROM ORDERS
 WHERE ORDER_DATE BETWEEN '2024-08-10' AND '2024-08-25'
 AND ORDER_STATUS != 'CANCELLED';
```

Result:

| | ORDER_ID | CUSTOMER_ID | ORDER_DATE | ORDER_STATUS | TOTAL_AMOUNT |
|---|---|---|---|---|---|
| 1 | 1004 | 101 | 2024-08-10 | Delivered | 3499.00 |
| 2 | 1006 | 105 | 2024-08-15 | Delivered | 5898.00 |
| 3 | 1007 | 106 | 2024-08-18 | Pending | 1299.00 |
| 4 | 1008 | 103 | 2024-08-20 | Delivered | 899.00 |
| 5 | 1009 | 107 | 2024-08-25 | Shipped | 6098.00 |

**Q11.** Explain what the index idx_orders_date does. How would it improve the performance of a query that filters orders by order_date? Write a sample query that would benefit from this index.

Answer:

- ✓ IDX_ORDERS_DATE : This is the index created on the column - (ORDER_DATE)
- ✓ An index helps SQL to locate rows faster.
- ✓ Without index SQL will scan the entire table.
- ✓ It will check every row one by one. Hence it will take more time.
- ✓ By using an index server performs an index seek.
- ✓ It directly locates required dates.
- ✓ Hence query executes faster.

Query:

```
SELECT * FROM ORDERS
WHERE ORDER_DATE = '2024-08-15';
```

Result:

| | ORDER_ID | CUSTOMER_ID | ORDER_DATE | ORDER_STATUS | TOTAL_AMOUNT |
|---|---|---|---|---|---|
| 1 | 1006 | 105 | 2024-08-15 | Delivered | 5898.00 |

Conclusion:

Indexes helps to optimize the search and improves query performance.

**Q12.** If you run: SELECT * FROM customers WHERE YEAR(join_date) = 2024; — would the index on join_date be used? Explain why or why not, and rewrite the query to be index-friendly (SARGable).

Answer:

- ✓ No index will not be used in these query.
- ✓ Because Query is not SARGable because YEAR() function is used in index column i.e. JOIN_DATE.
- ✓ So before SQL compare value with 2024, it has to calculate YEAR(JOIN_DATE) for every row.
- ✓ It makes the query more difficult.

SARGable query:

```
SELECT * FROM CUSTOMERS
WHERE JOIN_DATE BETWEEN '2024-01-01' AND '2024-12-31';
```

Result:

| | CUSTOMER_ID | FIRST_NAME | LAST_NAME | EMAIL | CITY | STATE | JOIN_DATE | IS_PREMIUM |
|---|---|---|---|---|---|---|---|---|
| 1 | 101 | Aarav | Sharma | aarav.s@email.com | Mumbai | Maharashtra | 2024-01-15 | 1 |
| 2 | 102 | Priya | Patel | priya.p@email.com | Ahme... | Gujarat | 2024-02-20 | 0 |
| 3 | 103 | Rohan | Gupta | rohan.g@email.co... | Delhi | Delhi | 2024-03-10 | 1 |
| 4 | 104 | Sneha | Reddy | sneha.r@email.co... | Hyder... | Telangana | 2024-04-05 | 0 |
| 5 | 105 | Vikram | Singh | vikram.s@email.c... | Jaipur | Rajasthan | 2024-05-12 | 1 |
| 6 | 106 | Ananya | Iyer | ananya.i@email.c... | Chennai | Tamil Nadu | 2024-06-18 | 0 |
| 7 | 107 | Karan | Mehta | karan.m@email.c... | Pune | Maharashtra | 2024-07-22 | 1 |
| 8 | 108 | Divya | Nair | divya.n@email.com | Kochi | Kerala | 2024-08-30 | 0 |

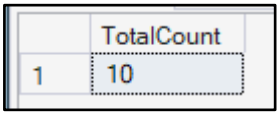
# Section C

**Q13.** Count the total number of orders in the orders table.

Query:

```
SELECT COUNT(*) AS TotalCount
FROM ORDERS;
```

Result:



| | TotalCount |
|---|---|
| 1 | 10 |

Observation:

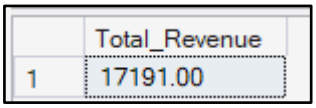
This query displays the total number of orders from the table by using the aggregate function i.e. COUNT().

**Q14.** Find the total revenue (SUM of total_amount) from all 'Delivered' orders.

Query:

```
SELECT SUM(TOTAL_AMOUNT) AS Total_Revenue
FROM ORDERS
WHERE ORDER_STATUS = 'Delivered';
```

Result:



| | Total_Revenue |
|---|---|
| 1 | 17191.00 |

Observation:

This query displays the sum of total amount from the ORDERS table by using the aggregate function i.e. SUM().

**Q15.** Calculate the average unit_price of products in each category.

Query:

```
SELECT CATEGORY, AVG(UNIT_PRICE) AS Average_category
FROM PRODUCTS
GROUP BY CATEGORY;
```

Result:

| | CATEGORY | Average_category |
|---|---|---|
| 1 | Clothing | 2699.000000 |
| 2 | Electronics | 2224.000000 |
| 3 | Home | 949.000000 |

Observation:

This query displays the average of unit price in each category from the PRODUCTS table by using the aggregate function i.e. AVG().

**Q16.** For each order status, find the count of orders and the total revenue. Sort the result by total revenue in descending order.

Query:

```
SELECT ORDER_STATUS,
COUNT (*) AS Total_Orders,
SUM(TOTAL_AMOUNT) AS Total_Revenue
FROM ORDERS
GROUP BY ORDER_STATUS
ORDER BY total_Revenue DESC;
```

Result:

| | ORDER_STATUS | Total_Orders | Total_Revenue |
|---|---|---|---|
| 1 | Delivered | 6 | 17191.00 |
| 2 | Shipped | 2 | 13596.00 |
| 3 | Cancelled | 1 | 2999.00 |
| 4 | Pending | 1 | 1299.00 |

Observation:

This query displays the total count of orders and total revenue by using the 2 aggregate functions i.e. COUNT() & SUM(). Also sorts the records by total revenue in descending order by using ORDER BY clause.

## **Q17.** Find the most expensive (MAX) and cheapest (MIN) product in each category.

Query:

```
SELECT CATEGORY,
MAX(UNIT_PRICE) AS Expensive_Category,
MIN(UNIT_PRICE) AS Cheapest_Category
FROM PRODUCTS
GROUP BY CATEGORY;
```

Result:

| | CATEGORY | Expensive_Category | Cheapest_Category |
|---|---|---|---|
| 1 | Clothing | 4599.00 | 799.00 |
| 2 | Electronics | 3499.00 | 899.00 |
| 3 | Home | 1299.00 | 599.00 |

Observation:

This query displays the most expensive product by using the aggregate function i.e. MAX() and most cheapest product by using the aggregate function i.e. MIN() from the PRODUCTS table.

The MIN() finds minimum value whereas MAX finds the maximum value from the table.

## **Q18.** List all product categories where the average unit_price is greater than ₹2000. (Hint: Use HAVING clause)

Query:

```
SELECT CATEGORY,
AVG(UNIT_PRICE) AS Average_Price
FROM PRODUCTS
GROUP BY CATEGORY
HAVING AVG(UNIT_PRICE) > 2000;
```

Result:

| | CATEGORY | Average_Price |
|---|---|---|
| 1 | Clothing | 2699.000000 |
| 2 | Electronics | 2224.000000 |

Observation:

This query finds the average of unit price by using the aggregate function i.e. AVG(). And then displays categories having average greater than 2000.

The GROUP BY clause groups the records based on categories and HAVING clause filters the GROUPS based on the condition.

# Section D

**Q19.** Write an INNER JOIN query to display each order along with the customer's first_name and last_name. Show: order_id, order_date, first_name, last_name, total_amount.

Query:

```
SELECT
O.ORDER_ID,
C.FIRST_NAME,
C.LAST_NAME,
O.ORDER_DATE,
O.TOTAL_AMOUNT
FROM ORDERS AS O
INNER JOIN CUSTOMERS AS C
ON O.CUSTOMER_ID = C.CUSTOMER_ID;
```

Result:

| | ORDER_ID | FIRST_NAME | LAST_NAME | ORDER_DATE | TOTAL_AMOUNT |
|---|---|---|---|---|---|
| 1 | 1001 | Aarav | Sharma | 2024-08-01 | 4498.00 |
| 2 | 1002 | Priya | Patel | 2024-08-03 | 799.00 |
| 3 | 1003 | Rohan | Gupta | 2024-08-05 | 7498.00 |
| 4 | 1004 | Aarav | Sharma | 2024-08-10 | 3499.00 |
| 5 | 1005 | Sneha | Reddy | 2024-08-12 | 2999.00 |
| 6 | 1006 | Vikram | Singh | 2024-08-15 | 5898.00 |
| 7 | 1007 | Ananya | Iyer | 2024-08-18 | 1299.00 |
| 8 | 1008 | Rohan | Gupta | 2024-08-20 | 899.00 |
| 9 | 1009 | Karan | Mehta | 2024-08-25 | 6098.00 |
| 10 | 1010 | Divya | Nair | 2024-08-28 | 1598.00 |

Observation:

This query joins the ORDER & CUSTOMER tables by using the INNER JOIN on the column CUSTOMER_ID. And then displays the customer's first name and last name from the CUSTOMERS table and order id, order date, total amount from ORDERS table.

**Q20.** Using a LEFT JOIN, list ALL customers and their orders (if any). Customers with no orders should still appear with NULL values for order columns.

Query:

```
SELECT
C.CUSTOMER_ID,
C.FIRST_NAME,
C.LAST_NAME,
O.ORDER_ID
FROM CUSTOMERS AS C
LEFT JOIN ORDERS AS O
ON C.CUSTOMER_ID = O.CUSTOMER_ID;
```

Result:

| | CUSTOMER_ID | FIRST_NAME | LAST_NAME | ORDER_ID |
|---|---|---|---|---|
| 1 | 101 | Aarav | Sharma | 1001 |
| 2 | 101 | Aarav | Sharma | 1004 |
| 3 | 102 | Priya | Patel | 1002 |
| 4 | 103 | Rohan | Gupta | 1003 |
| 5 | 103 | Rohan | Gupta | 1008 |
| 6 | 104 | Sneha | Reddy | 1005 |
| 7 | 105 | Vikram | Singh | 1006 |
| 8 | 106 | Ananya | Iyer | 1007 |
| 9 | 107 | Karan | Mehta | 1009 |
| 10 | 108 | Divya | Nair | 1010 |

Observation:

This query joins the CUSTOMER & ORDERS table by using LEFT JOIN on CUSTOMER_ID column. Left join returns the all rows from left table and only matching rows from right table.

So here as a left table CUSTOMER it returns everything i.e. customer id, first name, last name and only matching rows from ORDERS table i.e. Order id.

**Q21.** Write a query using JOINs across three tables (orders → order_items → products) to show: order_id, product_name, quantity, unit_price, and discount_pct for each order item.

Query:

```
SELECT
C.FIRST_NAME,
C.LAST_NAME,
P.PRODUCT_NAME,
OI.QUANTITY,
O.TOTAL_AMOUNT
FROM CUSTOMERS AS C
INNER JOIN ORDERS O
ON C.CUSTOMER_ID = O.CUSTOMER_ID
INNER JOIN ORDER_ITEMS AS OI
ON O.ORDER_ID = OI.ORDER_ID
INNER JOIN PRODUCTS AS P
ON OI.PRODUCT_ID = P.PRODUCT_ID;
```

Result:

| | FIRST_NAME | LAST_NAME | PRODUCT_NAME | QUANTITY | TOTAL_AMOUNT |
|---|---|---|---|---|---|
| 1 | Aarav | Sharma | Wireless Earbuds | 2 | 4498.00 |
| 2 | Aarav | Sharma | Laptop Stand | 1 | 4498.00 |
| 3 | Priya | Patel | Cotton T-Shirt | 1 | 799.00 |
| 4 | Rohan | Gupta | Smart Watch | 1 | 7498.00 |
| 5 | Rohan | Gupta | Running Shoes | 1 | 7498.00 |
| 6 | Aarav | Sharma | Bluetooth Speaker | 1 | 3499.00 |
| 7 | Sneha | Reddy | Smart Watch | 1 | 2999.00 |
| 8 | Vikram | Singh | Wireless Earbuds | 1 | 5898.00 |
| 9 | Vikram | Singh | Running Shoes | 1 | 5898.00 |
| 10 | Ananya | Iyer | Bedsheet Set | 1 | 1299.00 |
| 11 | Rohan | Gupta | Laptop Stand | 1 | 899.00 |
| 12 | Karan | Mehta | Bluetooth Speaker | 1 | 6098.00 |
| 13 | Karan | Mehta | Cushion Covers (... | 2 | 6098.00 |
| 14 | Divya | Nair | Bedsheet Set | 1 | 1598.00 |
| 15 | Divya | Nair | Cushion Covers (... | 1 | 1598.00 |

Observation:

This query first joins the CUSTOMER & ORDERS table by using INNER JOIN on CUSTOMER_ID column. Then it joins the ORDER & ORDER ITEM table by using the INNER JOIN on ORDER_ID column and lastly it joins the ORDER ITEM & PRODUCT table by using INNER JOIN on the PRODUCT_ID column.

So here we can display the records from all three tables.

## **Q22.** Explain the difference between LEFT JOIN and RIGHT JOIN with an example from this schema. When would you use a FULL OUTER JOIN?

Answer:

- Left join:

✓ Returns all the rows from left table.

✓ Returns only matching rows from right table.

✓ If there is no match, then right table contains NULL.

- Right join:

✓ Returns all rows from right table.
✓ Returns only matching rows from left table.
✓ If there is no match, then left table contains NULL.

- Full outer join:

✓ Returns all rows from both tables.
✓ Returns matching rows where available.
✓ Contains NULL where there is no match.

EX:

LEFT JOIN:

```
SELECT
C.CUSTOMER_ID,
C.FIRST_NAME,
O.ORDER_ID
FROM CUSTOMERS AS C
LEFT JOIN ORDERS AS O
ON C.CUSTOMER_ID = O.CUSTOMER_ID;
```

Result:

| | CUSTOMER_ID | FIRST_NAME | ORDER_ID |
|---|---|---|---|
| 1 | 101 | Aarav | 1001 |
| 2 | 101 | Aarav | 1004 |
| 3 | 102 | Priya | 1002 |
| 4 | 103 | Rohan | 1003 |
| 5 | 103 | Rohan | 1008 |
| 6 | 104 | Sneha | 1005 |
| 7 | 105 | Vikram | 1006 |
| 8 | 106 | Ananya | 1007 |
| 9 | 107 | Karan | 1009 |
| 10 | 108 | Divya | 1010 |

RIGHT JOIN

```
SELECT
C.CUSTOMER_ID,
C.FIRST_NAME,
O.ORDER_ID
FROM CUSTOMERS AS C
RIGHT JOIN ORDERS AS O
ON C.CUSTOMER_ID = O.CUSTOMER_ID;
```

Result:

| | CUSTOMER_ID | FIRST_NAME | ORDER_ID |
|---|---|---|---|
| 1 | 101 | Aarav | 1001 |
| 2 | 102 | Priya | 1002 |
| 3 | 103 | Rohan | 1003 |
| 4 | 101 | Aarav | 1004 |
| 5 | 104 | Sneha | 1005 |
| 6 | 105 | Vikram | 1006 |
| 7 | 106 | Ananya | 1007 |
| 8 | 103 | Rohan | 1008 |
| 9 | 107 | Karan | 1009 |
| 10 | 108 | Divya | 1010 |

**Q23.** Identify all Foreign Key relationships in the schema. Explain what would happen if you tried to insert an order with customer_id = 999 (which doesn't exist in customers).

Answer:

Foreign key:

1. ORDERS.CUSTOMER_ID -> CUSTOMERS.CUSTOMER_ID
2. ORDER_ITEMS.ORDER_ID -> ORDERS.ORDER_ID
3. ORDER_ITEMS.PRODUCT_ID -> PRODUCTS.PRODUCT_ID

What happen if we try to insert invalid customer?

- ✓ As the customer id 999 does not exists in the table SQL server will not able to find such record.

Hence :

- ✓ SQL will reject this insertion.
- ✓ will display the error message.

## Section E

**Q24.** Write a query using CASE to classify products into price tiers:

- 'Budget' → unit_price < 1000
- 'Mid-Range' → unit_price BETWEEN 1000 AND 3000
- 'Premium' → unit_price > 3000

Display: product_name, unit_price, price_tier.

Query:

```
SELECT
PRODUCT_NAME,UNIT_PRICE,
CASE
    WHEN UNIT_PRICE < 1000 THEN 'Budget'
    WHEN UNIT_PRICE BETWEEN 1000 AND 3000 THEN 'Mid-Range'
    WHEN UNIT_PRICE > 3000 THEN 'Premium'
END AS Price_Tier
FROM PRODUCTS;
```

Result:

| | PRODUCT_NAME | UNIT_PRICE | Price_Tier |
|---|---|---|---|
| 1 | Wireless Earbuds | 1499.00 | Mid-Range |
| 2 | Cotton T-Shirt | 799.00 | Budget |
| 3 | Smart Watch | 2999.00 | Mid-Range |
| 4 | Running Shoes | 4599.00 | Premium |
| 5 | Bluetooth Speaker | 3499.00 | Premium |
| 6 | Bedsheet Set | 1299.00 | Mid-Range |
| 7 | Laptop Stand | 899.00 | Budget |
| 8 | Cushion Covers (Set) | 599.00 | Budget |

Observation:

This query classifies the each record by using the filter conditions with the help of CASE.

CASE contains the multiple conditions separated by WHEN keyword.

This specifies whenever the condition becomes true it executes THEN part and exits.

**Q25.** Using a CASE statement inside an aggregate function, count how many orders are 'Delivered' vs 'Not Delivered' (all other statuses). Display the result in a single row.

Query:

```
SELECT
SUM( CASE
WHEN ORDER_STATUS = 'Delivered' THEN 1
ELSE 0
                END) AS Delivered_Orders,
SUM( CASE
WHEN ORDER_STATUS != 'Delivered' THEN 1
ELSE 0
                END) AS Not_Delivered_Orders
FROM ORDERS;
```

Result:

| | Delivered_Orders | Not_Delivered_Orders |
|---|---|---|
| 1 | 6 | 4 |

Observation:

This query classifies each record according to its order status as ‘Delivered ’ or ‘not delivered’ by using the filter conditions with the help of CASE.

Then it uses the aggregate function SUM() to add all the records return by the CASE statement.

Hence query returns the count of delivered and not delivered orders in one row.

## **Q26.** Explain each letter of ACID:

- A – Atomicity

- C – Consistency
- I – Isolation
- D – Durability

Give a real-world example (e.g., bank transfer) showing why each property is important.

Answer:

- Atomicity:
  - ✓ Transaction is treated as a single unit of work.
  - ✓ Either all operations are completed successfully or none of them are performed.
  - ✓ If any operation fails, entire transaction is rolled back.

  Ex:

  Suppose 1000 rupees transferred from Account A to account B.

  If 1000 rupees are debited from account A but fails to credit in account B .

  Then atomicity ensures debit operation is also rolled back.

- Consistency:
  - ✓ Consistency ensures that every transaction moves database from one valid state to another valid state while following all database rules.

  Ex:

  Suppose Account A has 5000 rupees and account B has 3000 rupees.

  Before transfer the total balance is 8000 rupees.

  After transferring 1000 rupees from Account A to account B.

  Hence after the transfer total balance is also 8000 rupees.

- Isolation:
  - ✓ It ensures that multiple transactions running simultaneously do not interfere with each other.

  Ex:

  Suppose two users attempt to withdraw money from same bank account at the same time.

  Isolation ensures that one transaction completes first. Then second transaction waits and then works with updated balance.

- Durability:
  - ✓ Durability ensures that once a transaction is committed, its changes are permanently stored in database and cannot be lost, even if system crashes.

  Ex:

  After a successful transfer of 1000 rupees, the transaction is committed.

  Even if server losses, when database restarts Account A still shows deducted amount. And Account B still shows credited amount.

  The committed transaction remains saved.

**Q27.** Write a SQL transaction that does the following atomically:

1. Insert a new order (order_id=1011, customer_id=102, today's date, 'Pending', 1598.00)
2. Insert two order items for that order
3. Update the stock_qty of the purchased products
4. If any step fails, ROLLBACK the entire transaction. Otherwise, COMMIT.

Write the complete BEGIN...COMMIT/ROLLBACK block.

Query:

```
BEGIN TRY
  BEGIN TRANSACTION;
  INSERT INTO ORDERS
  (ORDER_ID, CUSTOMER_ID, ORDER_DATE, ORDER_STATUS, TOTAL_AMOUNT)
  VALUES
  (1011, 102, GETDATE(), 'Pending', 1598.00);

  INSERT INTO ORDER_ITEMS
  (ITEM_ID, ORDER_ID, PRODUCT_ID, QUANTITY, DISCOUNT_PCT)
  VALUES
  (16, 1011, 201, 1, 10),
  (17, 1011, 202, 2, 5);

  UPDATE PRODUCTS
  SET STOCK_QTY = STOCK_QTY - 1
  WHERE PRODUCT_ID = 201;
```

```
    UPDATE PRODUCTS
    SET STOCK_QTY = STOCK_QTY - 2
    WHERE PRODUCT_ID = 202;

    COMMIT TRANSACTION;
    PRINT 'Transaction completed successfully.';
END TRY
BEGIN CATCH
    ROLLBACK TRANSACTION;
    PRINT 'Transaction failed.';
    PRINT ERROR_MESSAGE();
END CATCH;
```

Conclusion:

TRY and Catch ensures all database operations are executed as a single unit.

If every operation succeeds, the changes are permanently saved using COMMIT.

If any operation fails database restores to its previous state using ROLLBACK